La cartera, leída por conjuntos

Qué se cambió en la lectura de pendientes, qué se conserva y qué demostró la medición

En una frase. La cartera del profesional (la lista de pendientes, API-CAR-01) hacía, en cada página, cinco a siete consultas a la base por cada persona y área. Ahora hace una consulta por tipo de hecho para toda la cartera, acotada a lo que el PDP permitió. Los permisos, la auditoría, el contenido y el orden no cambian: lo comprueba una prueba que compara, byte a byte, la respuesta nueva con una copia congelada de la anterior.

Lo que midió (misma máquina, mismo proceso, las dos versiones alternadas): con 200 personas, la mediana de una página pasó de 2032 a 938 ms (−53.8 %); con historia, de 2350 a 968 ms (−58.8 %). Cero errores. El detalle, con sus condiciones y límites, en la lámina 5.

Para quién. Para Dirección: entender el mecanismo y aprender a pedir y evaluar este tipo de trabajo. Cada lámina responde una pregunta; la columna de la derecha trae los conceptos, con su nombre técnico y su traducción.

Qué es. Una explicación visual de un cambio de rendimiento acotado, no una guía nueva ni parte de la tesis.

De dónde sale. El código de `main` `e4512cb` (la versión anterior) y el PR #222 (la propuesta); la segunda medición de la cartera (`EVIDENCIA/RENDIMIENTO-CARTERA/CONCURRENCIA.md`) y la medición antes/después de esta tanda (`LECTURA-POR-CONJUNTOS.md`).

Cómo se arma. Fuente Typst y láminas generadas desde código con el motor de las guías: ningún diagrama es una imagen generativa. Solo datos sintéticos.

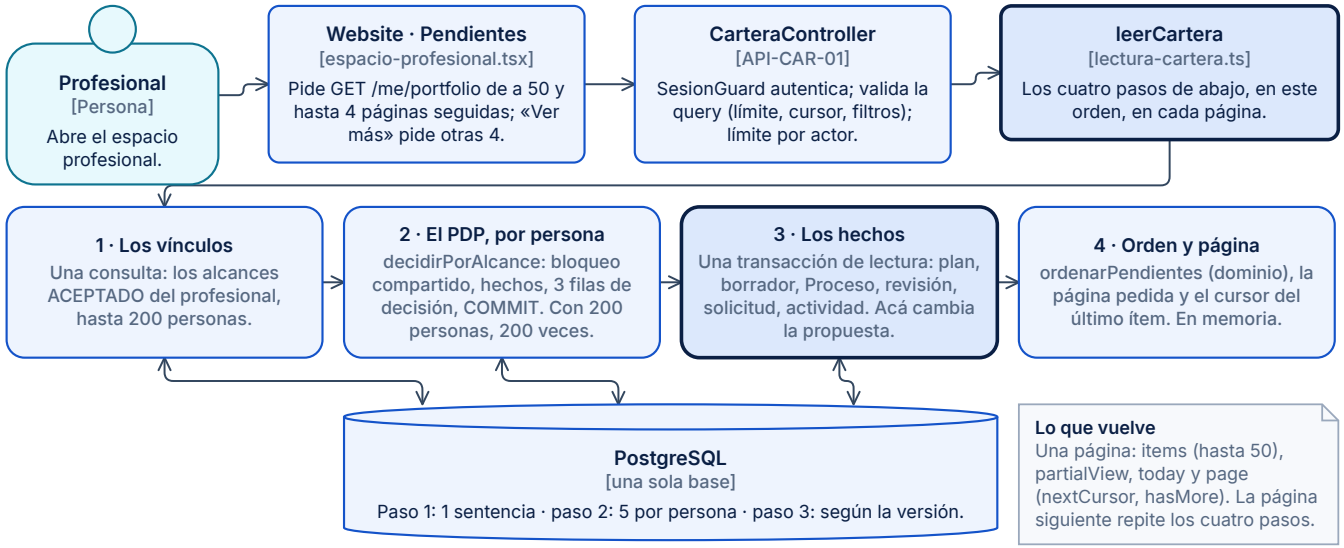
Para comprobar. Tres preguntas en la última lámina; las respuestas, aparte, en `RESPUESTAS.md`.

LÁMINAS

1 · ¿Cómo se construye la cartera, de la pantalla a la base? 2
2 · ¿Dónde se repetía trabajo? 3
3 · ¿Cómo funciona la propuesta y qué conserva? 4
4 · ¿Qué pasa con los permisos y las transacciones? 5
5 · ¿Qué demostraron las mediciones y qué sigue sin saberse? 6
6 · Para comprobar, y lo que queda como propuesta 7

1 · ¿Cómo se construye la cartera, de la pantalla a la base?

Un pedido de una página recorre el website, el controlador y cuatro pasos de la lectura. El website pide hasta cuatro páginas al abrir el espacio profesional, y cada una repite los cuatro pasos.



CA-01. El recorrido de una página. Los pasos 1 y 2 no cambian con la propuesta; el 3 es el que se lee por conjuntos; el 4 es en memoria.

- **Personas y pendientes no son lo mismo.** Una persona con Nutrición y Entrenamiento puede aportar cuatro pendientes; otra, ninguno. Por eso la paginación corta la lista de pendientes ya ordenada, no la lista de personas: paginar personas antes de calcular cambiaría el orden por urgencia y la cobertura de cada página.
- **Cada página repite todo.** La segunda página, pedida con el cursor, vuelve a decidir con el PDP por cada persona y a leer todos los hechos: cuesta lo mismo que la primera y deja las mismas filas de decisión. No lo cambia esta propuesta (ver lámina 6).

Para leer en el código: `apps/web/src/app/pro/espacio-profesional.tsx` · `apps/api/src/cartera/cartera.controller.ts` · `apps/api/src/cartera/lectura-cartera.ts`

CONCEPTO 1

Pendiente

En criollo. Algo que el profesional tiene que atender: una revisión vencida, un plan en borrador, una persona sin plan, un formulario sin responder, una evaluación en preparación.
En BE. Siete tipos (`TipoDePendiente`). No se guardan: se calculan en cada lectura a partir de hechos que ya existen.

CONCEPTO 2

PDP (punto de decisión)

En criollo. El único lugar que decide si este profesional puede ver a esta persona en esta área. Decide de nuevo en cada pedido y deja escrita cada decisión.
En BE. `PdpService.decidirPorAlcance`: una transacción por persona, con bloqueos compartidos, los hechos de las siete dimensiones y tres filas en `decision_de_acceso`.

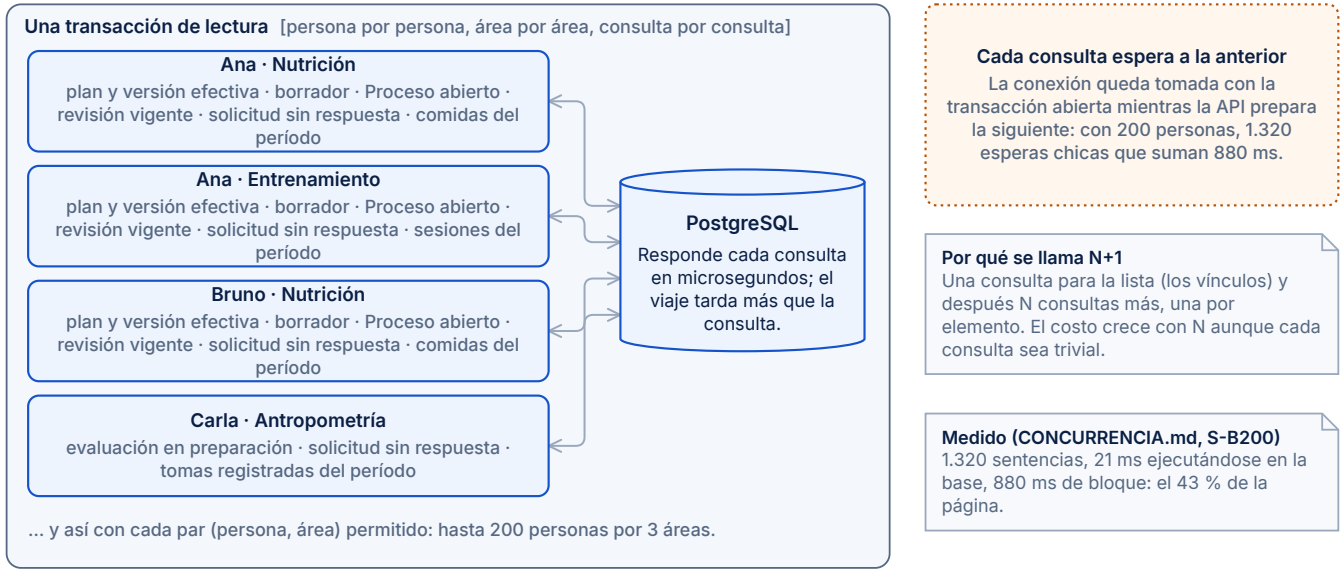
CONCEPTO 3

Cursor de paginación

En criollo. Un marcador opaco que dice «seguí después de este ítem».
En BE. La clave del último ítem servido (tipo, dominio, persona), en base64url. Si la lista cambió y la clave ya no está, 400 `INVALID_CURSOR`.

2 · ¿Dónde se repetía trabajo?

En el paso 3. La base ejecutaba cada consulta en microsegundos; lo caro era la cantidad de viajes de ida y vuelta, uno detrás del otro.



Cada consulta espera a la anterior
La conexión queda tomada con la transacción abierta mientras la API prepara la siguiente: con 200 personas, 1.320 esperas chicas que suman 880 ms.

Por qué se llama N+1
Una consulta para la lista (los vínculos) y después N consultas más, una por elemento. El costo crece con N aunque cada consulta sea trivial.

Medido (CONCURRENCIA.md, S-B200)
1.320 sentencias, 21 ms ejecutándose en la base, 880 ms de bloque: el 43 % de la página.

CA-02. Un ejemplo chico: tres personas y cuatro pares. Con 200 personas, el mismo dibujo tiene unos 300 pares.

Cartera	Pares (persona, área)	Consultas del paso 3, antes	Consultas del paso 3, después
3 personas del ejemplo	4	21 (6 ó 7 por par)	≈ 11
50 personas, perfil base	75	≈ 500	≈ 11
200 personas, perfil base	300	≈ 1.320 (medido)	≈ 11
200 personas con historia	350	≈ 2.000 (medido)	≈ 11

Para leer en el código: `test/integration/soporte-cartera-anterior.ts (la copia congelada de la lectura anterior)`

CONCEPTO 4

Ida y vuelta (round trip)

En criollo. Mandar una consulta y esperar su respuesta. Aunque la consulta sea trivial, el viaje tiene un costo fijo; mil viajes chicos tardan más que uno grande.
En BE. Con 200 personas, el paso 3 hacía 1.320 viajes que sumaban 21 ms de ejecución en PostgreSQL y 880 ms de reloj (CONCURRENCIA.md).

CONCEPTO 5

N+1

En criollo. El patrón de «una consulta para la lista y después una por cada elemento». El costo crece con la cantidad de elementos aunque cada consulta sea rápida.
En BE. Una consulta de vínculos y, por cada par (persona, área), cinco a siete consultas más: el plan y su versión efectiva, el borrador, el Proceso, la expectativa, la solicitud y la actividad.

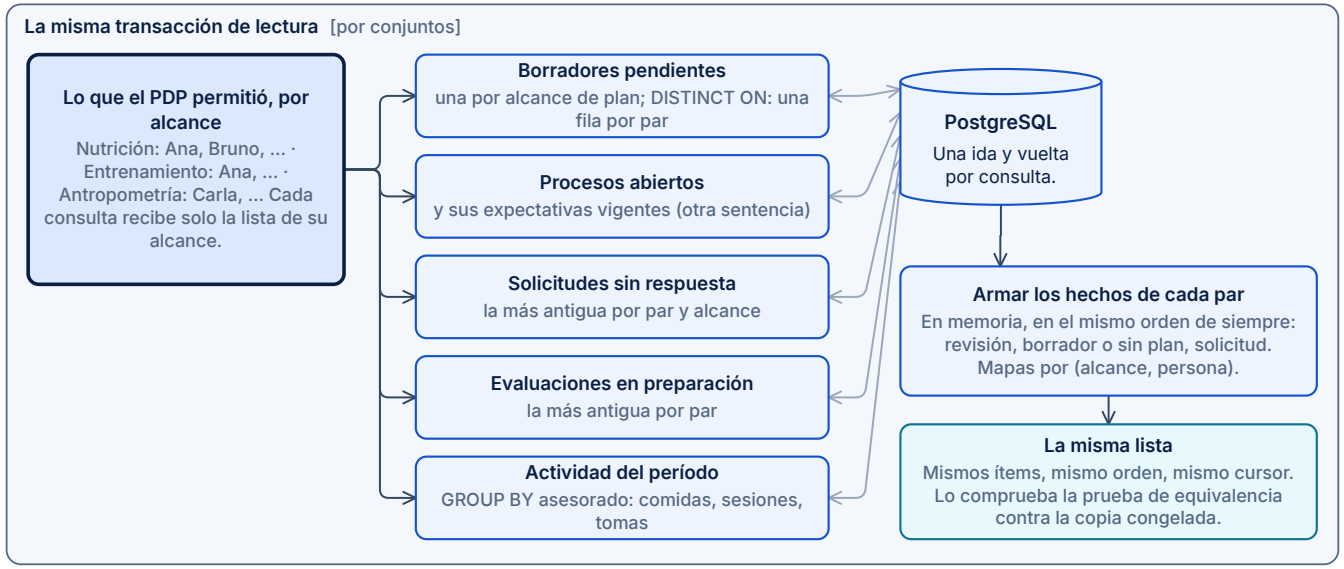
CONCEPTO 6

Transacción abierta esperando

En criollo. La conexión a la base queda ocupada mientras la API piensa qué pedir después; nadie más la usa en ese rato.
En BE. `idle_in_transaction_time`: cerca de la mitad del tiempo de una página con 200 personas era la conexión esperando a la API.

3 · ¿Cómo funciona la propuesta y qué conserva?

Pedir de una vez, para toda la cartera, cada tipo de hecho; después, armar en memoria los hechos de cada par, en el mismo orden de siempre.



CA-03. La lectura por conjuntos. Las cinco consultas de la izquierda son, en total, unas once sentencias; la lista final es la misma.

Qué conserva, y cómo se sabe. El contrato (`CarteraResponse`), el contenido, el orden (`ordenarPendientes`, el mismo, sobre la misma lista de entrada), los filtros, el límite y el cursor; los permisos (el paso 2 no cambia); la auditoría (las mismas tres filas por persona y pedido); y una sola transacción de lectura, más corta. Una prueba de integración compara, en cada consulta posible, la respuesta nueva con la de la copia congelada de la lectura anterior: tienen que ser iguales byte a byte, también después de revocar un consentimiento, pausar, reanudar o finalizar un vínculo. Otra cuenta las sentencias de un pedido con 6 y con 12 personas: la diferencia tiene que ser exactamente la del PDP.

Para leer en el código: `apps/api/src/cartera/lectura-cartera.ts` · `apps/api/src/proceso/proceso.service.ts` · `test/integration/cartera-por-conjuntos.int-spec.ts`

CONCEPTO 7

Consulta por conjuntos

En criollo. Pedir de una vez los datos necesarios de varias personas autorizadas, en lugar de pedirlos de a una.
En BE. `leerHechosPorConjuntos`: cada consulta recibe la lista de asesorados que el PDP permitió **en ese alcance** (`ids[alcance]`). Lo no permitido no se lee: no se trae de más para filtrar después.

CONCEPTO 8

DISTINCT ON

En criollo. «De cada grupo, dame solo la primera fila según este orden». Devuelve una fila por par aunque haya muchos borradores.
En BE. El borrador pendiente de cada plan: el más antiguo posterior a la activación vigente, una consulta por alcance de plan.

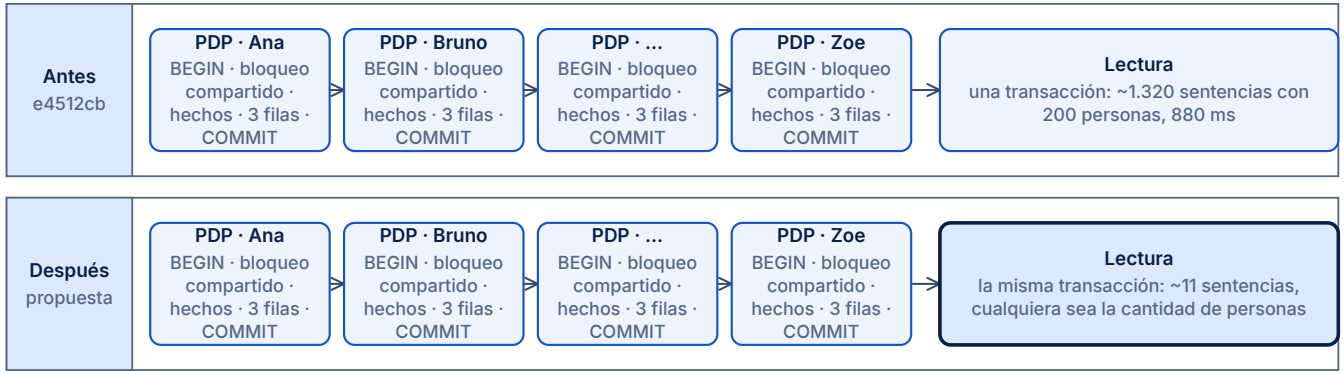
CONCEPTO 9

GROUP BY

En criollo. Agrupar filas por una clave y resumir cada grupo: cuántas hay y cuál es la última.
En BE. La actividad del período por asesorado: comidas sin anular, sesiones y tomas registradas, con `_count` y `_max`.

4 · ¿Qué pasa con los permisos y las transacciones?

Nada cambia en cómo se decide ni en cuándo. La decisión sigue siendo por persona, en su propia transacción, antes de leer; la lectura nunca decidió nada.



Un corte en el medio (revocar, pausar, suspender)
Toma en modo exclusivo las mismas filas que la decisión toma en modo compartido: espera a que termine la decisión en curso, y la decisión siguiente ya lo ve. Eso es el paso 2, que no cambia.

La lectura nunca decide
Lee solo los pares ya permitidos. Si un corte llega durante la lectura, esa página ya estaba decidida (antes y después); la próxima página vuelve a decidir. Lo único que cambia es que la lectura dura menos.

CA-04. Las dos versiones, una arriba de la otra: el paso 2 es idéntico; solo la lectura se acorta.

Lo que sí cambiaría una garantía, y por eso queda como propuesta
Decidir las 200 personas en **una sola** transacción (la alternativa 2 de CONCURRENCIA.md) ahorraría 199 confirmaciones al disco, pero retendría los bloqueos compartidos de toda la cartera durante toda la decisión: un corte que llegue en el medio esperaría más, y las 200 decisiones tendrían el mismo instante. Es una decisión de Dirección sobre la pieza de autorización (08 §27), no una optimización.

Para leer en el código: `apps/api/src/autorizacion/pdp.service.ts` · `prisma/concurrencia.ts` · `prisma/migrations/20261003000000_pdp_bloqueos_en_una_llamada`

CONCEPTO 10

Transacción

En criollo. Un paquete de operaciones que vale entero o no vale: o se confirma todo (COMMIT) o se deshace todo.
En BE. Cada decisión del PDP es una transacción: si no se puede registrar la decisión, no hay acceso (UC-I02 E06). La lectura de hechos es otra transacción, solo de lectura.

CONCEPTO 11

Bloqueo compartido y exclusivo

En criollo. Compartido: «estoy mirando esto, no me lo cambies». Exclusivo: «lo voy a cambiar, esperen». Varios compartidos conviven; un exclusivo espera a todos.
En BE. `be_bloquear_lo_que_corta` toma en modo compartido las filas que pueden cortar el acceso; revocar, pausar, finalizar o suspender las toman en modo exclusivo. Una decisión queda antes o después de cada corte, nunca en el medio.

CONCEPTO 12

READ COMMITTED

En criollo. El nivel de aislamiento por omisión de PostgreSQL: cada sentencia ve lo confirmado hasta ese instante.
En BE. La transacción de lectura siempre fue READ COMMITTED: con menos sentencias adentro, la foto es más corta en el tiempo, no una garantía distinta.

5 · ¿Qué demostraron las mediciones y qué sigue sin saberse?

Las dos versiones corrieron alternadas en la misma máquina de GitHub (4 CPU (AMD EPYC 7763 64-Core Processor), 15.61 GB, Node v22.23.2, PostgreSQL 16.15), el mismo proceso y las mismas carteras, en la corrida `40692787787`. Cada lote comprobó qué versión tenía puesta.

Escenario (200 personas salvo aclaración)	Pedidos	p50 base	p50 prop.	Δ p50	p95 base	p95 prop.	Δ p95	Ruido base	Pág./s base → prop.
50 personas	100	528	241	**−54.4 %**	562	257	−54.3 %	−6.5 %	—
100 personas	100	1035	473	**−54.3 %**	1073	503	−53.1 %	−0.2 %	—
200 personas	100	2032	938	**−53.8 %**	2216	1040	−53.1 %	−9.5 %	—
200 con historia	100	2350	968	**−58.8 %**	2644	1036	−60.8 %	+1.4 %	—
1 a la vez, sin pausa	100	2019	917	**−54.6 %**	2102	969	−53.9 %	+2.2 %	0.5 → 1.08
3 a la vez, sin pausa	300	3174	1461	**−54 %**	3578	1836	−48.7 %	0 %	0.93 → 2.01
5 a la vez, sin pausa	500	4801	2175	**−54.7 %**	5608	2712	−51.6 %	+10.5 %	1.06 → 2.28

Latencias en ms, medidas en el cliente, las dos repeticiones juntas. «Ruido»: cuánto difirió la base entre sus dos repeticiones (p50). Los Δ por repetición están en `LECTURA-POR-CONJUNTOS.md`.

Dónde cambió el tiempo (200 personas, medianas por pedido). Bloque de lectura: 1137 → 23 ms (perfil base), 1383 → 48 ms (con historia). Bloque del PDP: 898 → 924 ms (no cambia por diseño). Sentencias por pedido: 6228 → 4915 (base), 7048 → 5067 (historia). COMMIT por pedido: 201 → 201. Filas de decisión por pedido: 600 a 600 → 600 a 600. **Lo que lee el website al abrir** (las primeras cuatro páginas, A-B200): 8227 → 3854 ms (−53.2 %). No es el tiempo de pantalla: el navegador no se midió. Errores en toda la corrida: 0. Inventario de pendientes de las 8 carteras, con las dos versiones: idéntico.

Lo que sigue sin saberse. Nada de Render (otra CPU, la base en otra máquina y la red en el medio). Cuántos profesionales reales aguanta: los clientes de C3 y C5 piden sin pausa. Más de cinco a la vez, escrituras simultáneas, meses de historia, el tiempo en el navegador. Y el paso 2 cuesta lo mismo: cada página con 200 personas sigue dejando 600 filas de decisión y 200 confirmaciones al disco.

Para leer en el código: `EVIDENCIA/RENDIMIENTO-CARTERA/LECTURA-POR-CONJUNTOS.md` · `test/rendimiento/cartera-ab.rendimiento.ts` · `test/rendimiento/analizar-ab.mjs`

CONCEPTO 13

p50 y p95

En criollo. La mediana (la mitad de los pedidos tardó menos) y el percentil 95 (el 95 % tardó menos): el caso típico y la cola.
En BE. La hipótesis decía que la cola (el PDP) bajaría menos que la mediana. No fue así: bajó igual, porque la cola de la lectura también pesaba.

CONCEPTO 14

Bucle cerrado

En criollo. Cada cliente pide la página siguiente apenas recibe la anterior, sin pausa. Mide cómo se reparte la máquina, no cuántas personas reales aguanta.
En BE. C1, C3 y C5: uno, tres y cinco profesionales, cada uno con su cartera de 200.

CONCEPTO 15

Variación entre repeticiones

En criollo. Cuánto difiere la misma versión consigo misma en dos corridas: el piso de ruido. Una mejora menor que ese piso no es una mejora.
En BE. Cada escenario corrió dos veces por versión; la columna «ruido» es la diferencia de la base entre sus dos repeticiones.

6 · Para comprobar, y lo que queda como propuesta

Tres preguntas para saber si se entendió el mecanismo (las respuestas, aparte, en `RESPUESTAS.md`), y las dos alternativas que no se implementaron porque tocan una garantía o un contrato.

Tres preguntas

1. Un profesional tiene 120 personas en Nutrición, 40 de ellas también en Entrenamiento. ¿Cuántas transacciones del PDP hace una página antes y después del cambio? ¿Y cuántas sentencias, aproximadamente, hace el paso 3 antes y después?
2. Una persona permitida en Nutrición y denegada en Entrenamiento (por ejemplo, con el consentimiento de Entrenamiento revocado). ¿En qué consultas del paso 3 aparece su identificador, y en cuáles no? ¿Por qué eso importa más que «filtrar después»?
3. Si alguien propusiera «calcular la cartera una vez y servir las páginas siguientes de lo calculado», ¿qué regla del legajo chocaría y qué pasaría con una revocación entre la página 1 y la página 2?

Lo que queda como propuesta (decisión de Dirección)

- **Las 200 decisiones en una transacción.** Ahorraría 199 COMMIT y la mayor parte del paso 2, pero cambia la ventana de los bloqueos y el instante de cada decisión. Antes de decidirlo: una prueba de carrera con PostgreSQL real (revocación durante la decisión) y la lectura de 08 §27.
- **No repetir la cartera en cada página.** Cuatro opciones, todas con un cambio de contrato, de pantalla o de regla: que el website pida solo lo que necesita y cuente con otra lectura liviana; subir el `limit` máximo (hoy 50); servir las páginas siguientes de lo calculado (choca con decidir en cada pedido sin caché, 08 §27.3); paginar por personas antes de calcular (cambia el orden por urgencia).
- **Qué se registra por lectura.** Tres filas por persona y por página, también en áreas sin vínculo. No se recomienda tocarlo para ganar velocidad: el legajo exige registrar cada decisión (08:634).

Cómo pedir y evaluar un trabajo así

Pedir: la hipótesis y el criterio de «suficiente» escritos **antes** de medir; la versión anterior como oráculo; las dos versiones en la misma máquina y alternadas; las muestras completas, no las mejores corridas. Evaluar: ¿la respuesta es idéntica?, ¿las filas de auditoría son las mismas?, ¿la mejora supera el ruido entre repeticiones?, ¿se movió el cuello de botella a otro lado?, ¿qué queda sin medir?